Tổng quan về
Cấu trúc dữ liệu & Giải thuật

Cấu trúc dữ liệu & Giải thuật – SW102DV01

(Data Structures & Algorithms)

Vũ Đình Khôi

Nội dung

- Khái niệm về cấu trúc dữ liệu và các kiểu dữ liệu
- Khái niệm về giải thuật và biểu diễn giải thuật
- Độ phức tạp của giải thuật
  - Khái niệm và cơ sở toán học
- Cách đánh giá độ phức tạp của giải thuật

Tham khảo

- Chapter 4. Robert Sedgewick. (1998). *Algorithms in C, Part 1-4, Third Edition*. Addison-Wesley
- Chapter 4. Robert Sedgewick. (2003). *Algorithms in Java, Part 1-4, Third Edition*. Addison-Wesley
- Chapter 1. Jeffrey J. McConnell. (2001). *Analysis of Algorithms: An Active Learning Approach*. Jones and Bartlett Publishers
- Chapter 1.4. Robert Sedgewick & Kevin Wayne. (2011). *Algorithms, Fourth Edition*. Addison-Wesley

Giới thiệu

I find that I don't understand things unless I try to program them.

Donald E. Knuth

Professor Emeritus at Stanford University

If you want to **master something**, teach it

-- Richard Feynman

Data types

- Dữ liệu (data)
  - Tập hợp các giá trị (value) của một biến (variable)
  - Mỗi dữ liệu có thể chứa nhiều giá trị gọi là “data item” hoặc “sub-item”
  - Ở mức độ trừu tượng cao hơn là các “entity”, “instance”, hoặc “object” của lớp.
- Kiểu dữ liệu
  - Ràng buộc (constraints) các giá trị mà một biến có thể lưu giữ
  - Định nghĩa các hành động/phép toán (operations) có thể thực hiện trên các giá trị đó.
- Các kiểu dữ liệu cơ bản (basic data types)
  - Primitive: int, char, float, boolean...
  - Các ngôn ngữ lập trình thường cung cấp sẵn
  - Số lượng bộ nhớ được cấp phát phụ thuộc vào ngôn ngữ lập trình hoặc các yếu tố khác
- Kiểu dữ liệu người dùng định nghĩa (User-defined data)
  - struct, class...

Abstract data types (ADTs)

- Ví dụ cần lưu thông tin tọa độ của một điểm (point) trên mặt phẳng/hệ tọa độ 2 chiều thì sao?
  - Sử dụng 2 biến số thực x, y (hoặc a, b) để lưu.
- Ở mức trừu tượng (abstraction) mang tính tổng quát hơn → có kiểu dữ liệu Point để biểu diễn thông tin tọa độ?
  - Kiểu dữ liệu Point này về bản chất vẫn dựa trên 2 biến số thực x, y
  - Người dùng (client program) chỉ biết đến kiểu Point và các thao tác/hành động tính toán trên kiểu Point như khoảng cách Euclid giữa các điểm... Mà không cần quan tâm đến sự phức tạp bên trong kiểu Point → người dùng chỉ giao tiếp thông qua các *interface* của lớp Point.
  - Point được gọi là kiểu dữ liệu trừu tượng (ADT)

Abstract data types (ADTs)

- Kiểu dữ liệu trừu tượng (ADT)
  - Một kiểu dữ liệu trừu tượng là kiểu dữ liệu (một tập hợp các giá trị và các thao tác trên các giá trị đó) chỉ được truy xuất (accessed) thông qua một interface.
- Trong C
  - Mảng các số nguyên phải được khai báo/xác định trước kích thước/chiều dài/số lượng các phần tử.
  - Muốn khắc phục nhược điểm trên → ADT linked-list chẳng hạn...
- C string library
  - Không phải là ADT vì chương trình/người dùng biết được chính xác chuỗi được biểu diễn (bằng mảng các ký tự) và có thể truy cập trực tiếp các ký tự thông qua chỉ mục (index) hoặc con trỏ (pointer).

Abstract data types (ADTs)

- ADT nhằm tách biệt (seperate) mức quan niệm (conceptual) mà chương trình thực hiện trên dữ liệu của chúng ta khỏi việc biểu diễn và triển khai các thuật toán của kiểu dữ liệu/cấu trúc dữ liệu đó.
  - Các bài sau, khi cài đặt các cấu trúc dữ liệu cụ thể như linked-list, stack, queue, tree... bằng nhiều kỹ thuật khác nhau, chúng ta sẽ hiểu kỹ ý nghĩa này.
- Ví dụ
  - Máy tính không có kiểu dữ liệu số ảo (complex number)
  - Làm sao biểu diễn *imaginary* number: $i^2 = -1 \text{ hay } i = \sqrt{-1}$
  - → trừu tượng
  - Làm sao biểu diễn đa thức (polinomial ADT) và các phép toán trên đa thức

$$a_3x^3 + a_2x^2 + a_1x + a_0$$

  - → trừu tượng

Data structures

- Cấu trúc dữ liệu (Data structure)
  - Gồm các tổ chức (organization) dữ liệu hoặc “item” có cấu trúc phức tạp hơn.
  - Triển khai thực tế (actual implemetation) của ADT trên máy tính (hoặc bộ nhớ).
  - Việc sử dụng các CTDL nên xem xét đến việc sử dụng bộ nhớ (memory) và thời gian (time) xử lý trên các cấu trúc này → Độ phức tạp (Complexity)
- Ví dụ
  - Stack ADT: gồm các hành động/thao tác push, pop, is_empty theo nguyên tắc LIFO (vào sau ra trước) nhưng không chỉ ra cách thực hiện cụ thể.
  - Stack data structure: cài đặt cụ thể một Stack ADT, có thể dùng array hoặc linked-list để cài đặt.

Data structures

- Tập trung vào cách tổ chức dữ liệu (organization)
- Có thể mô hình (modeled) bằng nhiều cách khác nhau. Ví dụ, các số có thứ tự (ordered numbers) có thể được lưu
  - Mảng tuyến tính (linear array): đơn giản, nhưng tốn thời gian thêm (insert) hoặc xóa (delete) dữ liệu
  - hoặc cấu trúc cây (tree-like structure): hơi phức tạp nhưng dễ dàng thêm, xóa
- Bất kỳ mô hình nào được sử dụng, phải
  - Biểu diễn mối liên hệ giữa các thành viên dữ liệu (data members)
  - Đơn giản (simple), hiệu quả (effective), và hiệu suất (efficient)

Data structures

- Một số CTDL được đề cập trong nội dung môn học
  - Mảng (Array)
  - Mảng 2 chiều (Two-dimensional arrays)
  - Danh sách liên kết (Linked list)
  - Ngăn xếp (Stacks)
  - Hàng đợi (Queue)
  - Đồ thị (Graph)
  - Cây (Tree)
- Các thao tác/hành động (operations) được thực hiện trên mỗi CTDL
  - Duyệt hay Liệt kê (Traversal)
  - Thêm (Inserting)
  - Xóa (Deleting)
  - Tìm kiếm (Searching)

Algorithm (Thuật toán/thuật giải/giải thuật)

- Richard Buckland (University of New South Wales) thường dùng từ rút gọn PAPP khi giảng dạy môn CTDL & GT
  - P = Problem
  - A = Algorithm
  - P = Program
  - P = Process
- Giải thuật/thuật toán (algorithm)?
  - Dãy các bước để hoàn thành công việc [trong thực tế] hoặc giải quyết vấn đề/bài toán [trên máy tính]
- Cần xem xét 2 yếu tố (theo thứ tự ưu tiên) sau đây
  - Tính chính xác (Correct)
  - Tính hiệu quả (Efficient)

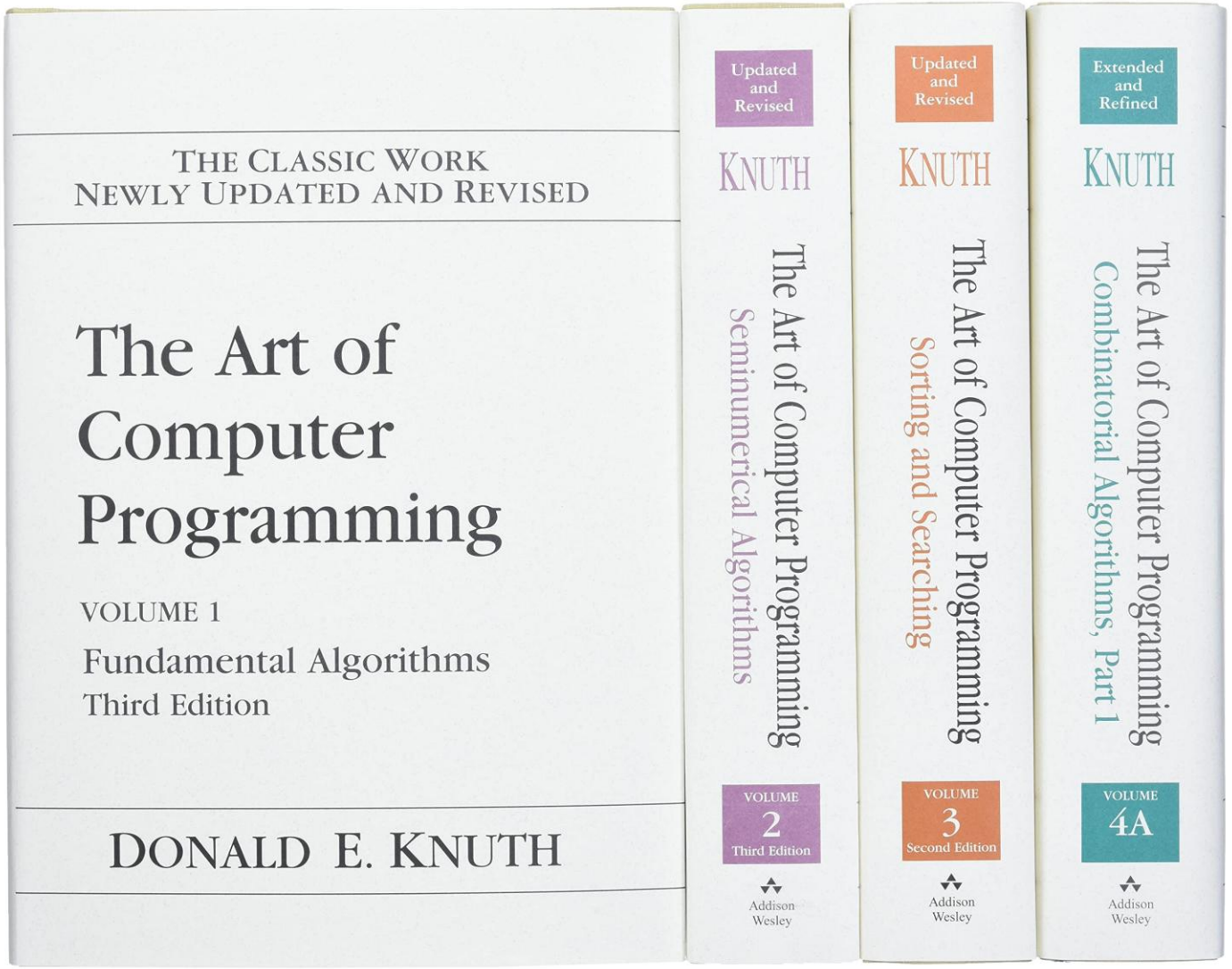
Algorithm

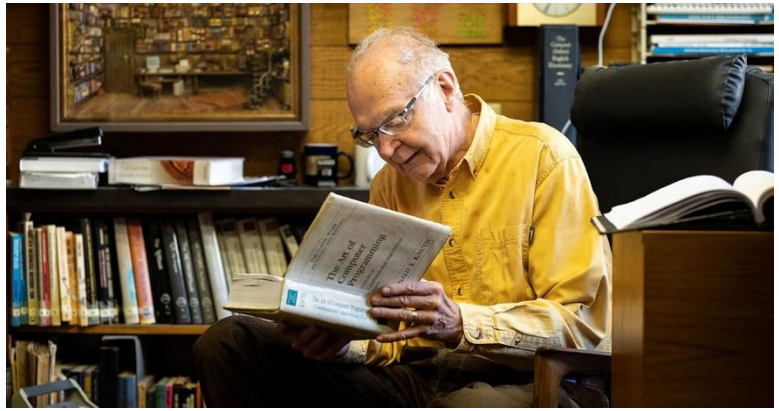
- Thuật toán (theo Cormen. Introduction to Algorithms, 4th)
  - Là phương pháp để giải quyết một vấn đề, một bài toán nào đó.

Informally, an ***algorithm*** is any well-defined computational procedure that takes some value, or set of values, as ***input*** and produces some value, or set of values, as ***output*** in a finite amount of time. An algorithm is thus a sequence of computational steps that transform the input into the output.

You can also view an algorithm as a tool for solving a well-specified ***computational problem***. The statement of the problem specifies in general terms the desired input/output relationship for problem instances, typically of arbitrarily large size. The algorithm describes a specific computational procedure for achieving that input/output relationship for all problem instances.

Algorithm

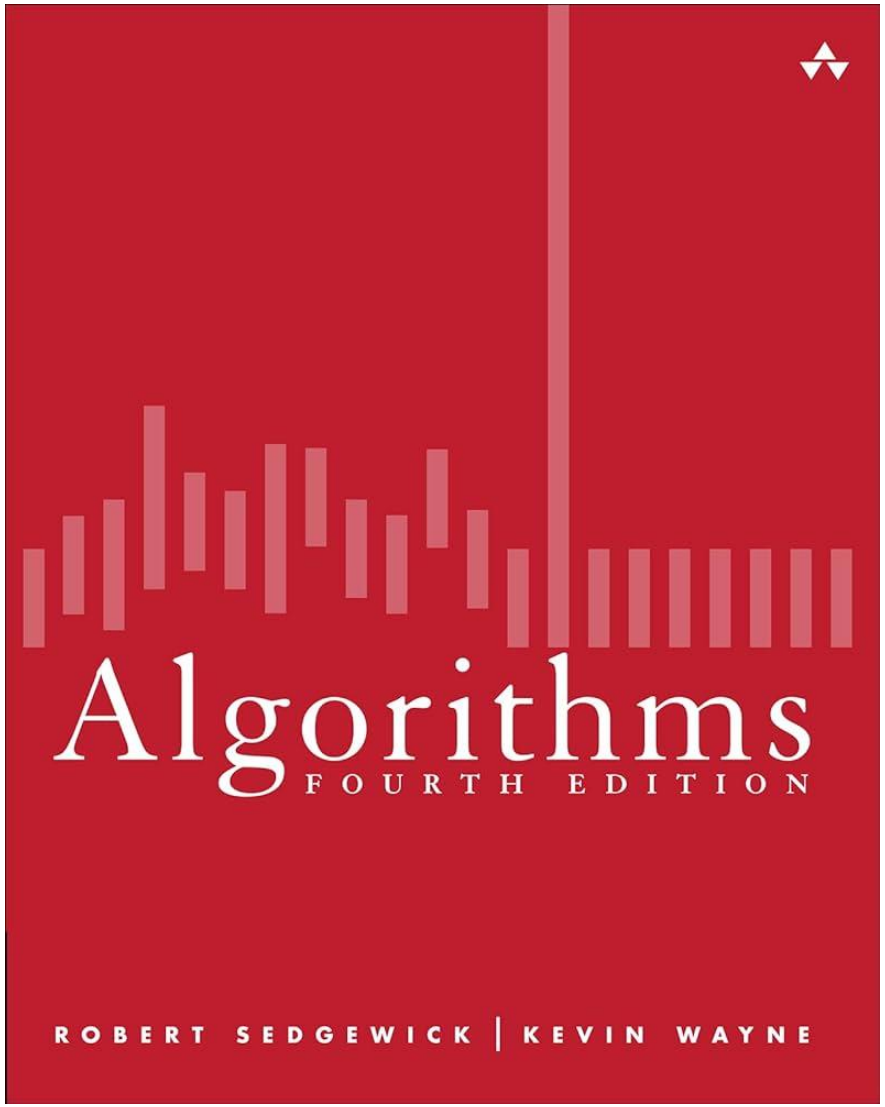


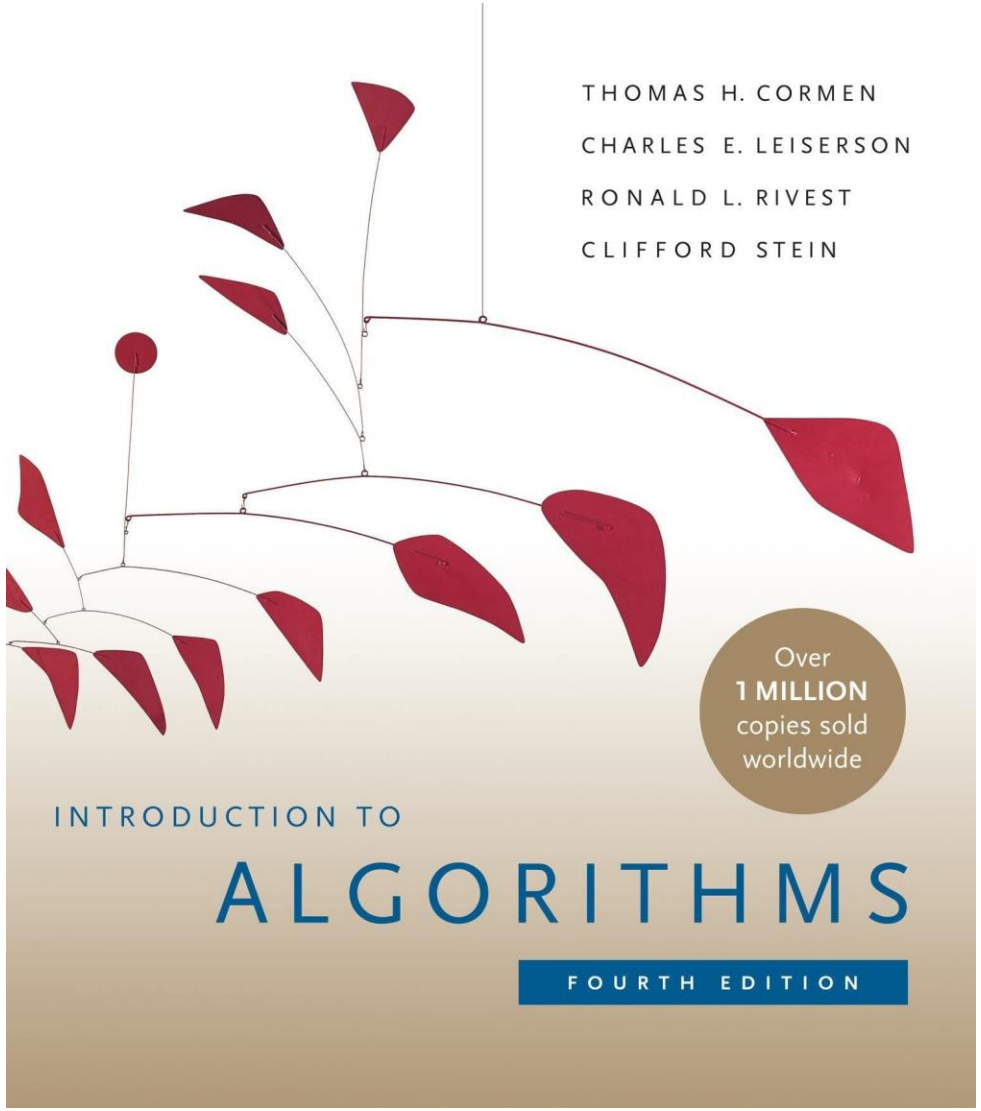


I find that I don't understand things unless I try to program them.

Donald E. Knuth
Professor Emeritus at Stanford University

Algorithm


Algorithms
FOURTH EDITION
ROBERT SEDGEWICK | KEVIN WAYNE


THOMAS H. CORMEN
CHARLES E. LEISERSON
RONALD L. RIVEST
CLIFFORD STEIN
Over
1 MILLION
copies sold
worldwide
INTRODUCTION TO
ALGORITHMS
FOURTH EDITION

Algorithm

- Các đặc tính của thuật toán
  - Tính khả thi
    - Phát biểu được → lập trình được
  - Tính hữu hạn
    - Số bước thực hiện phải hữu hạn
    - Thời gian thực hiện phải hữu hạn
  - Tính xác định
    - Kết quả thực hiện (output) tại mỗi bước
  - Tính chính xác
    - Với mỗi input → output
  - Tính hiệu quả
    - Thực thi với thời gian tối ưu/tiết kiệm bộ nhớ
    - Thiết kế "đẹp"

Algorithm

- Bài toán
  - Bài toán gốc, bài toán thực tế
- Thuật toán
  - Cách giải quyết [trên máy tính] bài toán thực tế
- Chương trình
  - Cài đặt hay thể hiện của thuật toán với ngôn ngữ lập trình cụ thể
- Mô hình tư duy máy tính
  - Abstraction: trừu tượng hóa/mô tả bài toán
  - Automation: lời giải/mô tả thuật toán
  - Analysis: phân tích được thuật toán/lời giải

Algorithm

- Biểu diễn thuật toán
  - Cách thức mô tả thuật toán nhằm giúp con người/máy tính hiểu và thực thi dễ dàng.
- Một số cách
  - Ngôn ngữ tự nhiên
  - Mã giả (pseudo code)
  - Lưu đồ (flowchart)
  - Biểu thức/phương trình toán học

Algorithm

- Một vài cách thiết kế thuật toán (tiếp cận để giải quyết vấn đề)
  - Giải thuật tham lam (Greedy approach)
  - Kỹ thuật Chia để trị (Divide and Conquer)
  - Quy hoạch động (Dynamic programming)
  - Kỹ thuật quay lui (Backtracking)

Giải thuật tham lam (Greedy approach)

- Bài toán đổi tiền/thối tiền
  - Giả sử có các tờ tiền mệnh giá (denominations): 500 , 100, 50, 20, 10, 5, 2, 1
  - Muốn đổi/thối lại 657, bạn chọn các mệnh giá nào để tổng số tờ (minimum number of bills/notes) là ít nhất?
- Lưu trữ dữ liệu bài toán như thế nào?
  - Cấu trúc dữ liệu (Data structure)
- Giải quyết bài toán thế nào?
  - Giải thuật (Algorithm)

Giải thuật tham lam (Greedy approach)

- Cấu trúc dữ liệu (Data structure)
  - Input: danh sách các mệnh giá tiền (linear array), số tiền cần đổi (int)
  - Output: danh sách số lượng tờ tiền của mỗi mệnh giá (linear array)
- Giải thuật (Algorithm)
  - Ưu tiên đổi tờ tiền có mệnh giá cao nhất trước (tham lam)
  - Sau đó chọn tờ có mệnh giá cao kế tiếp....
  - Cho đến khi đổi hết số tiền
- Nhận xét giải thuật tham lam
  - Chọn cách tốt nhất (cục bộ) tại thời điểm đang xét
  - Cách tiếp cận tốt nhưng không chắc luôn luôn đi tới giải pháp tối ưu (optimal solution)

Giải thuật tham lam (Greedy approach)

```c
// Giải thuật tham lam (Greedy approach)
// Bài toán đổi tiền (note_change.c)
// Đầu vào: Số tiền cần đổi, mảng chứa các loại tiền
#include <stdio.h>

void note_change(int money, int notes[]) {
    int quantity = 0;
    int i = 0;
    while (money > 0) {
        quantity = money / notes[i];
        if (quantity > 0) printf("%d note(s) of %d\n", quantity, notes[i]);
        money = money % notes[i];
        i++;
    }
}
```

Giải thuật tham lam (Greedy approach)

```
void main() {
    int money = 657;
    int notes[] = {500, 100, 50, 20, 10, 5, 2, 1};
    int n = sizeof(notes) / sizeof(notes[0]);
    note_change(money, notes);
}
```

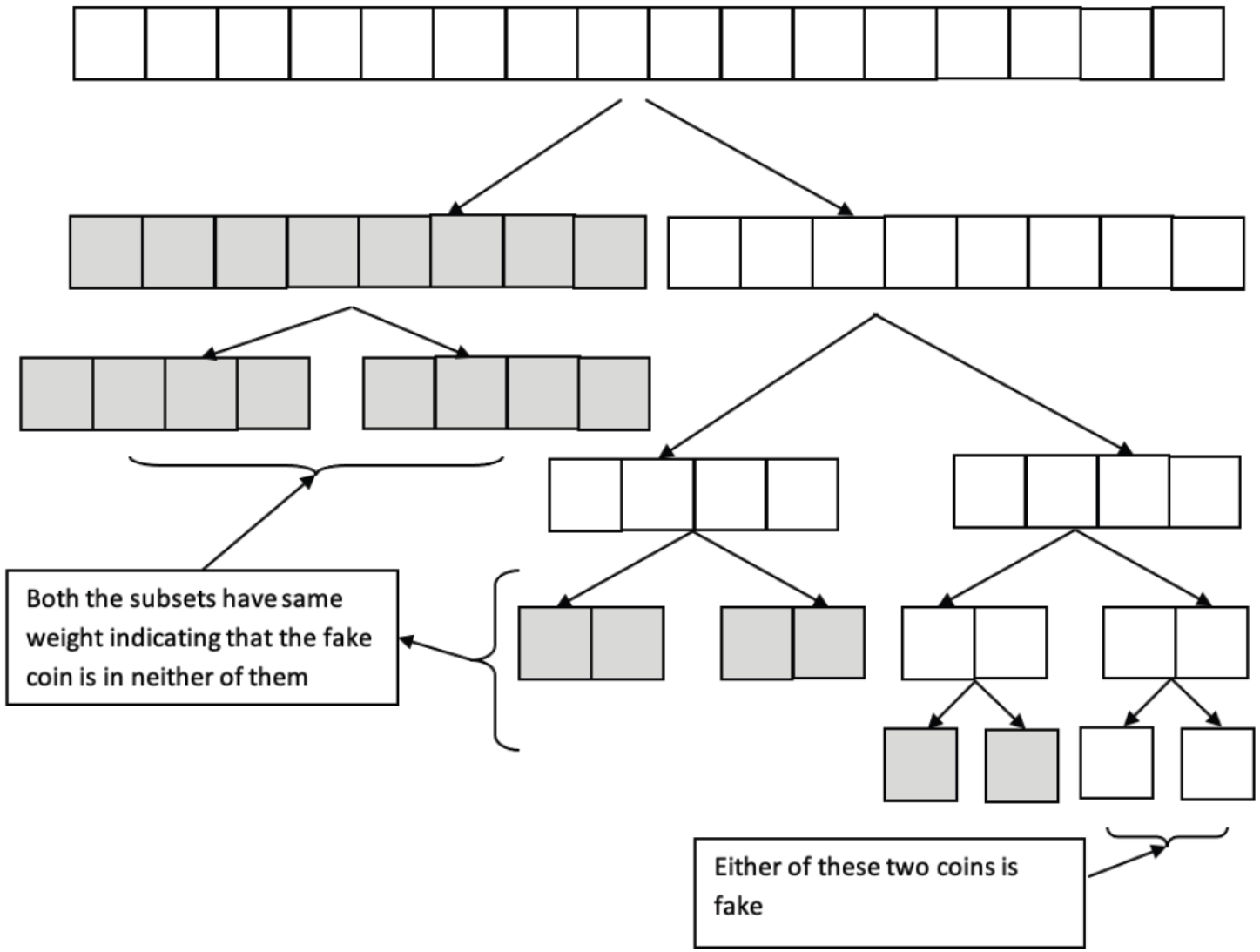
```
1 note(s) of 500
1 note(s) of 100
1 note(s) of 50
1 note(s) of 5
1 note(s) of 2
```

Kỹ thuật chia để trị (Divide and conquer)

- Bài toán tìm đồng xu giả (Fake coin problem)
  - Giả sử có n đồng xu (coins) với $n = 2^k$
  - Trong n đồng xu có duy nhất một đồng xu giả, có khối lượng khác (nhẹ hơn hoặc nặng hơn) so với (n-1) đồng xu thật còn lại.
  - Làm sao tìm ra đồng xu giả trong n đồng xu?
- Thuật toán
  - Chia n đồng xu làm 2 phần (mỗi phần n/2 đồng xu).
    - Nếu 2 phần bằng nhau ➔ không có đồng xu giả
    - Ngược lại ➔ có đồng xu giả
  - Tiếp tục, mỗi phần chia làm 2 phần nhỏ (mỗi phần n/4 đồng xu)
    - So sánh 2 phần trong từng phần nhỏ, nếu bằng nhau ➔ xu giả nằm trong 2 phần còn lại
  - Cuối cùng, chỉ còn 2 phần (mỗi phần 1 đồng xu)
    - Lấy 1 đồng xu (trong phần bất kỳ) để so sánh với đồng xu thật trong (n-2) đồng xu còn lại.

Kỹ thuật chia để trị (Divide and conquer)

- Ví dụ chia để trị
  - Chia bài toán thành các bài toán con (sub-problem) cùng loại
  - Giải bài toán con?
- Trong ví dụ này, mỗi lần chia thành các tập con n/2, n/4, n/8... phần tử
  - Lần thứ k là: $n/2^k$
  - Lần cuối cùng tập con chỉ còn 1 phần tử
  - $n/2^k = 1$ hay $n = 2^k$
  - Nên $k = log_2(n)$



Kỹ thuật chia để trị (Divide and conquer)

- Ví dụ chia để trị
  - Chia bài toán thành các bài toán con (sub-problem) cùng loại
  - Giải bài toán con?
- Trong ví dụ này, mỗi lần chia thành các tập con n/2, n/4, n/8... phần tử
  - Lần thứ k là: $n/2^k$
  - Lần cuối cùng tập con chỉ còn 1 phần tử
  - $n/2^k = 1$ hay $n = 2^k$
  - Nên $k = log_2(n)$
- Kỹ thuật này có thể áp dụng cho các bài toán dạng tìm kiếm (problems like searching) nếu mảng được sắp xếp (sorted).

Kỹ thuật chia để trị (Divide and conquer)

```c
// Thuật toán Chia để trị (Divide and Conquer)
// Bài toán tìm đồng xu giả (fake_coin.c)
#include <stdio.h>

// Cân các đồng xu và trả về tổng khối lượng
int weight(int coins[], int start, int end) {
    int sum = 0;
    for (int i = start; i < end; i++) {
        sum += coins[i];
    }
    return sum;
}

```

Kỹ thuật chia để trị (Divide and conquer)

```c
// Tìm vị trí đồng xu giả trong mảng coins từ vị trí start đến end
void fake_coin(int coins[], int start, int end) {
    // chỉ có 1 đồng xu duy nhất: trả về index của đồng xu đó
    if (start == end - 1) {
        printf("Fake coin may be at position %d\n", start);
        return;
    }
    int mid = (start + end) / 2;
    // Nếu khối lượng 2 phần khác nhau, đồng xu giả nằm trong 2 phần này
    if (weight(coins, start, mid) != weight(coins, mid, end)) {
        fake_coin(coins, start, mid);
        fake_coin(coins, mid, end);
    }
    return;
}
```

Kỹ thuật chia để trị (Divide and conquer)

```
30 void main() {
31     int n = 16;
32     int coins[n];
33
34     for (int i = 0; i < n; i++) {
35         coins[i] = 10;
36     }
37     coins[3] = 9;  // vị trí đồng xu giả
38
39     fake_coin(coins, 0, n);
40 }
```

Bài tập:
Viết lại chương trình với khối lượng các đồng xu là kiểu float

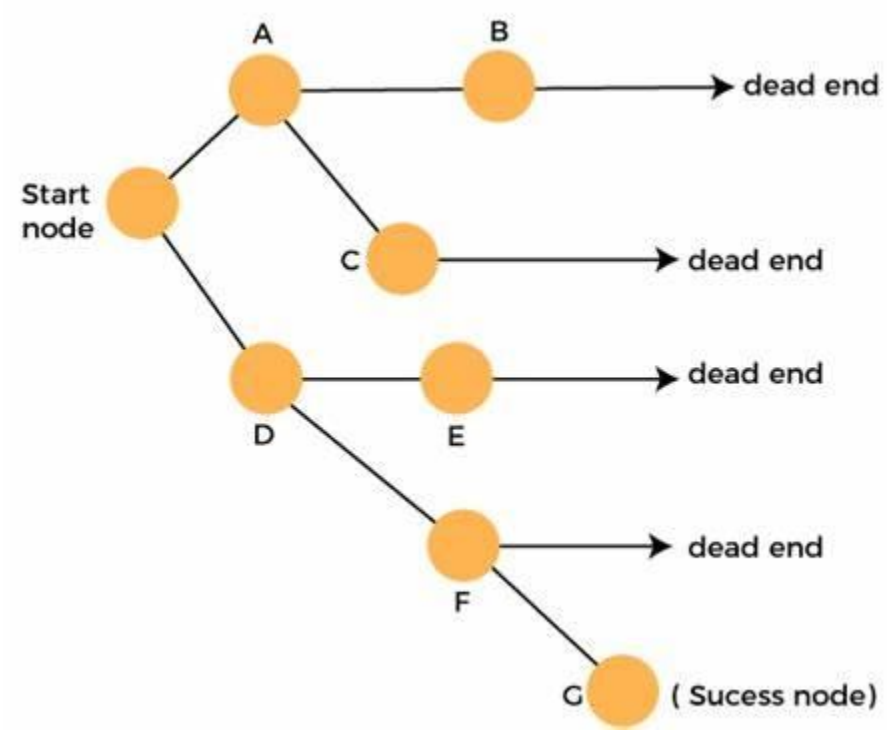
```
Fake coin may be at position 2
Fake coin may be at position 3
```

Kỹ thuật quay lui (Backtracking)

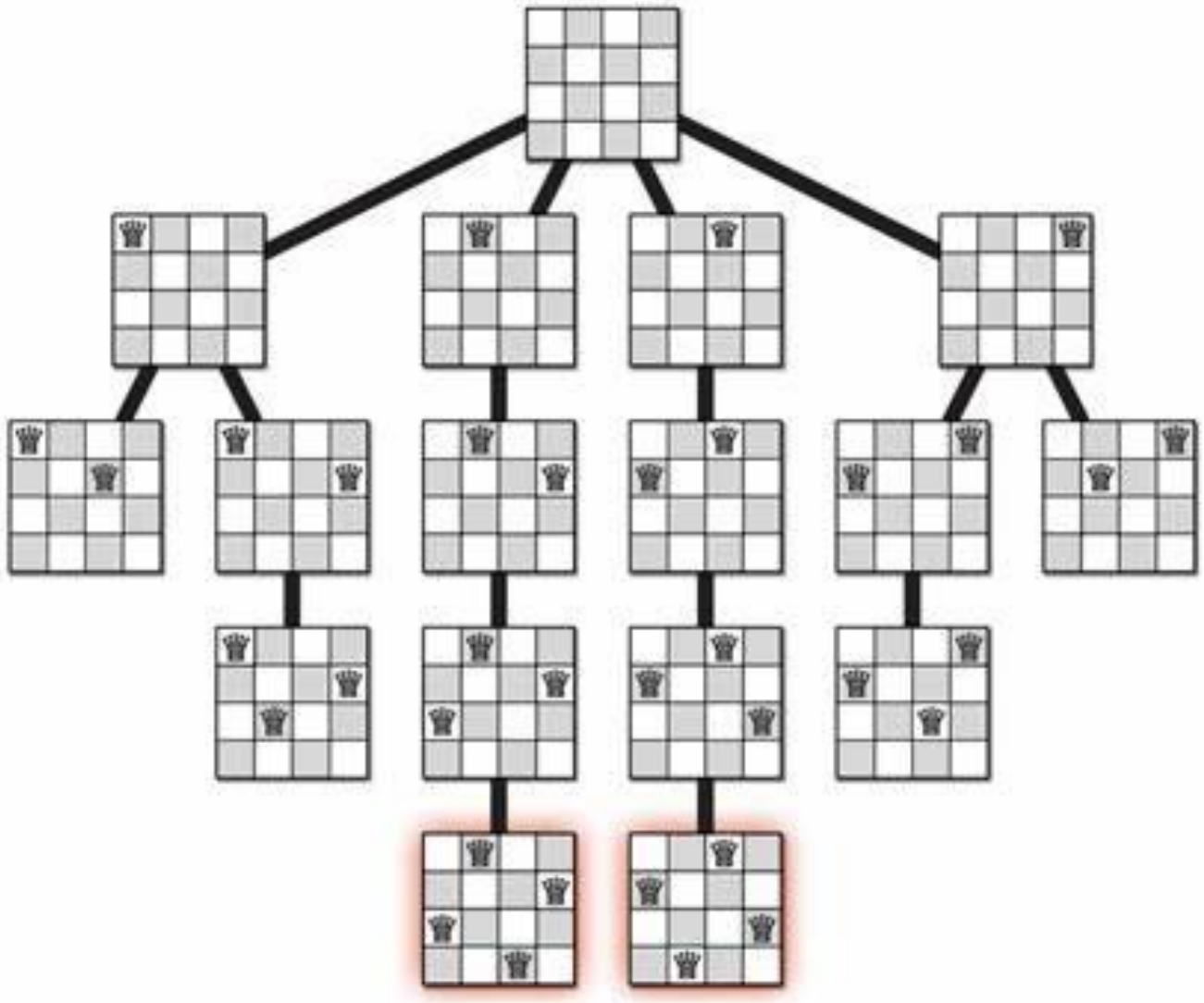
- Tại mỗi bước [lặp/tính toán/tìm lời giải] của bài toán
  - Tìm cách loại bỏ các lựa chọn (options/choices) không thể đưa đến lời giải
  - Và tiếp tục xử lý các lựa chọn tiềm năng (potential) đưa đến lời giải
- Cài đặt (implement) kỹ thuật quay lui tương đối phức tạp.
- Ví dụ các bài toán
  - 8 hậu (8-Queens problem)
  - Sudoku
  - Tô màu đồ thị (Graph coloring problems)
- Đệ quy (Recursion)
  - Là một phần của kỹ thuật quay lui
  - Gọi lại/giải quyết bài toán tương tự bài toán ban đầu nhưng với không gian nhỏ hơn.
  - Cài đặt đơn giản

Kỹ thuật quay lui (Backtracking)

- Tên gọi khác
  - Vét cạn (Brute Force)
  - Thử và sai (Trial and error)



Kỹ thuật quay lui (Backtracking)



Quy hoạch động (Dynamic programming)

- Chia bài toán thành các bài toán nhỏ hơn (smaller sub-problems):
  - Giải các bài toán nhỏ
  - Kết quả của các bài toán nhỏ dùng để tính toán (construct) hoặc đóng góp (contribute) cho bài toán lớn hơn (*Điểm này khác với Thuật toán chia để trị*).
- Kỹ thuật quy hoạch động hiệu quả hơn các kỹ thuật quay lui (backtracking).

Quy hoạch động (Dynamic programming)

```c
#include <stdio.h>
#include <time.h>

// Tính phần tử thứ n trong dãy Fibonacci (fibonacci_2.c)
// Sử dụng Quy hoạch động (Dynamic programming)
// Không sử dụng đệ quy/Khử đệ quy (Removing recursion)
int fibonacci(int n) {
    // Sử dụng mảng để lưu kết quả tính toán của các phần tử trước
    // 2 phần tử đầu tiên là 1, 1
    int L[500] = {1, 1};
    int i;
    for (i = 2; i < n; i++) {
        L[i] = L[i - 1] + L[i - 2];
    }
    return L[n - 1];
}

```

Giới thiệu

- Khi phân tích thuật toán, đầu tiên phải kiểm tra được tính đúng đắn
  - Giải quyết đúng yêu cầu bài toán, nếu không thì tính hiệu quả (efficiency) không còn quan trọng.
- Phân tích thuật toán?
  - Xác định số lượng "thời gian" thực thi/chạy thuật toán.
  - "Thời gian" ở đây KHÔNG phải là giây hay đo lường theo đồng hồ, mà là xấp xỉ (approximation) số lượng các thao tác tính toán (operations) mà thuật toán thực hiện.
- Phân tích thuật toán nhằm tìm một phương trình (equation)?
  - Mô tả mối liên hệ giữa số lượng các thao tác tính toán và quy mô/kích thước (size) của [dữ liệu] đầu vào (input).

Giới thiệu

- Ví dụ bài toán tìm kiếm tuần tự phần tử X trong một dãy số cho sẵn, số lần so sánh:
  - Trường hợp tốt nhất (best case): 1
  - Trường hợp xất nhất (worst case): n
  - Trường hợp trung bình (average case): (n + 1) / 2
- Hợp lý nhất là dùng ước lượng thời gian thực hiện trung bình T_{aver} để so sánh đánh giá thuật giải.
- Nếu tính thời gian thực hiện trung bình quá khó khăn, có thể đánh giá căn cứ vào trường hợp xấu nhất, tức là dùng T_{max}

Giới thiệu

- Giả sử để giải cùng một bài toán, có hai giải thuật giải T_1, T_2 với thời gian thực hiện tương ứng là:
  - $T_1(n) = C_1 * n$
  - $T_2(n) = C_2 * n^2$
- C_1, C_2 là các hằng số → khi n đủ lớn (vô cùng lớn) chắc chắn $T_2(n)$ sẽ lớn hơn $T_1(n)$, bất chấp C_1 có thể lớn hơn C_2 nhiều lần. Ví dụ:
  - $T_1(n) = 1000 * n$
  - $T_2(n) = 10 * n^2$
  - So sánh T_1 và T_2 khi n = 1, n = 10, n = 100...?
- Trong lĩnh vực các "vô cùng lớn", các hằng số không quan trọng, độ lớn phụ thuộc chủ yếu vào tốc độ tăng của T(n) khi n tăng.

Giới thiệu

- Về tổng thể, các thuật toán có thể chia thành 2 nhóm/lớp chính
  - Thuật toán lặp (repetitive)
  - Thuật toán đệ quy (recursive)
- Các thuật toán lặp
  - Dựa trên các vòng lặp (loops) và câu lệnh điều kiện (conditional statements)
- Các thuật toán đệ quy
  - Giải bài toán lớn bằng cách chia nhỏ và áp dụng cùng thuật toán cho các phần nhỏ này.
  - Còn gọi là các thuật toán Chia để trị (divide and conquer algorithms).

Ví dụ: xét 2 thuật toán đơn giản sau

```
int find_max_1(int a, int b, int c, int d) {
    int largest = a;
    if (b > largest) largest = b;
    if (c > largest) largest = c;
    if (d > largest) largest = d;
    return largest;
}

int find_max_2(int a, int b, int c, int d) {
    if (a > b) {
        if (a > c) {
            if (a > d)
                return a;
            else
                return d;
        } else {
            if (c > d)
                return c;
            else
                return d;
        }
    } else {
        if (b > c) {
            if (b > d)
                return b;
            else
                return d;
        } else {
            if (c > d)
                return c;
            else
                return d;
        }
    }
}
```

Ví dụ: xét 2 thuật toán đơn giản sau

- Thuật toán nào hiệu quả hơn?
  - *find_max_1()* dễ đọc, dễ hiểu.
  - Số lượng phép so sánh cần thực hiện để tìm số lớn nhất trong 4 số là như nhau (time complexity).
  - Nhưng về không gia lưu trữ (space) thuật toán *find_max_1()* kém hơn vì cần biến tạm (temporary variable) *Largest*.
- Vậy phân tích thuật toán là gì? (What is analysis?)
  - Cung cấp thông tin nền tảng để chúng ta có thể hình dung thuật toán thực thi bao lâu cho một tập hợp bài toán/vấn đề cho trước?
  - Mục tiêu là để so sánh hiệu quả hai thuật toán khác nhau để giải cùng một bài toán.

Các lớp [dữ liệu] đầu vào (Input classes)

- Xét đoạn chương trình tìm số lớn nhất trong 10 số khác nhau từ 0-9

```c
int find_max_3(int list[], int n) {
    int largest = list[0];
    for (int i = 1; i < n; i++) {
        if (list[i] > largest) largest = list[i];
    }
    return largest;
}

void main() {
    // tìm max trong danh sách 10 số phân biệt từ 0-9
    int list[10] = {6, 5, 2, 3, 7, 1, 8, 9, 0, 4};
    int max = find_max_3(list, 10);
    printf("Max: %d\n", max);
}
```

Input classes

- Xét dữ liệu đầu vào (input) của hàm *find_max_3(int list[], int n)*
- Có bao nhiêu *List[10]* chứa 10 số nguyên khác nhau từ 0-9?
  - 10! = 3,628,800
- Lưu ý 1
  - Thuật toán chỉ cần tìm ra số lớn nhất (ví dụ số 9).
- Trong 3,628,800 chia ra làm 10 lớp (classes) input, mỗi lớp input gồm
  - 362,880 tập đầu vào (input set) có số 9 ở vị trí đầu tiên list[0]
  - 362,880 tập đầu vào (input set) có số 9 ở vị trí thứ hai list[1]
  - ...
  - 362,880 tập đầu vào (input set) có số 9 ở vị trí cuối cùng list[9]
- Lưu ý 2
  - Đối với mỗi tập đầu vào → hàm *find_max_3()* xử lý giống nhau (cùng số thao tác)

Input classes

- Kết luận
  - Xác định các lớp đầu vào (input classes)
  - Tìm hiểu thuật toán thực thi thế nào trên mỗi input từ mỗi lớp
  - Nếu chia lớp đúng → Tất cả các input trên cùng một lớp sẽ có cùng số lượng thao tác tính toán (same number of operations).

Space complexity

- Độ phức tạp về không gian lưu trữ
  - Một số thuật toán không chỉ xem xét tính hiệu quả về thời gian (time complexity) mà còn tính đến cần thêm bao nhiêu không gian lưu trữ (extra space) để hoàn thành công việc xử lý của thuật toán.
- Ví dụ, để lưu các số thực với độ chính xác 0.1 (one place of precision after the decimal point) từ -10 đến +10
  - Nếu dùng kiểu số thực (real number): mỗi số cần từ 4-8 bytes
  - Nếu nhân các số trên cho 10 → dãy từ -100 đến +100: lưu thành các số nguyên 1 bytes
  - Cách nào hiệu quả hơn?

What to count and consider?

- Có 2 lớp thao tác tính toán (operators) chính cần xem xét
  - Các phép toán so sánh (comparison operators)
    - ==, !=, <, >, <=, >=
    - Trong các bài toán tìm kiếm (searching) và sắp xếp (sorting)
  - Các phép toán số học (arithmetic operators) gồm 2 nhóm con
    - Phép cộng (additive): + (addition) , - (subtraction), ++ (increment), -- (decrement)
    - Phép nhân (multiplicative): * (multiplication), / (division), % (modulus)
    - Phép nhân tốn nhiều thời gian hơn so với phép cộng
- Trường hợp đặc biệt, nhân/chia một số nguyên với số 2^n
  - Sử dụng phép dịch bit trái/phải (left/right shift operator) → nhanh hơn nhiều lần so với thực hiện phép nhân trực tiếp
  - a * 8 tương đương a << 3

Cases to consider

- Các trường hợp của dữ liệu đầu vào (input) làm cho thuật toán thực hiện nhanh nhất hoặc chậm nhất?
- Best case
  - Thường là giá trị hằng số nhỏ → do đó ít khi quan tâm
- Worst case
  - Trường hợp phân tích quan trọng nhất, vì nó liên quan đến thời gian tối đa (most time) thuật toán thực thi.
- Average case
  - Trường hợp khá khó để phân tích vì có nhiều thông số chi tiết, cần:
    - Chia tập input thành các lớp (input classes)
    - Xác định xác suất (probability) của input trong từng nhóm
    - Thời gian thực thi của thuật toán trên từng nhóm

Average case

- Công thức tính thời gian trung bình

$$A(n) = \sum_{i=1}^{m} p_i \times t_i$$

  - n: kích thước (size) của input
  - m: số input classes
  - p_i: xác suất để một input rơi vào nhóm i
  - t_i: thời gian thuật toán thực thi cho một input thuộc nhóm i
- Ví dụ, xét thuật toán `find_max_3(int list[], int n)`
  - Có 10 input classes (m = 10)
  - Xác suất để một input thuộc vào các nhóm là như nhau: $p_i = 1/10 = 0.1$

$$A(n) = \frac{1}{m} \sum_{i=1}^{m} t_i$$

Mathematical background

- Giá trị sàn (floor) hay cận dưới của một số
  - $floor(x)$ hay $\lfloor x \rfloor$: số nguyên lớn nhất nhưng nhỏ hơn hay bằng x
  - Ví dụ: $\lfloor 2.5 \rfloor = 2$; $\lfloor -7.3 \rfloor = -8$
- Giá trị trần (ceiling) hay cận trên của một số
  - $ceiling(x)$ hay $\lceil x \rceil$: số nguyên nhỏ nhất nhưng lớn hơn hay bằng x
  - Ví dụ: $\lceil 2.5 \rceil = 3$; $\lceil -7.3 \rceil = -7$
- Vì chúng ta hay sử dụng các số dương (positive numbers)
  - floor: như truncation (làm tròn xuống, cắt bỏ phần dư)
  - ceiling: như rounding up (làm tròn lên)

Mathematical background

- Logarithms

$$a^b = c \text{ tương đương } b = \log_a c$$
$$\text{nếu } x > y \text{ thì } \log_b x > \log_b y$$

- Viết tắt
  - $\log_{10} a \rightarrow \log a$
  - $\log_2 a \rightarrow \lg a$
- Logarithms là một hàm tăng chặt chẽ (Logarithms are a strictly increasing function)
  - Nếu so sánh a= 10 với b = 10,000 ta thấy b tăng gấp 1000 lần a
  - Nếu so sánh a $= \log 10 = 1$ với b $= \log 10000 = 4$ ta chỉ thấy b tăng gấp 4 lần a

Mathematical background

- Logarithms

$$\log_B 1 = 0$$

$$\log_B B = 1$$

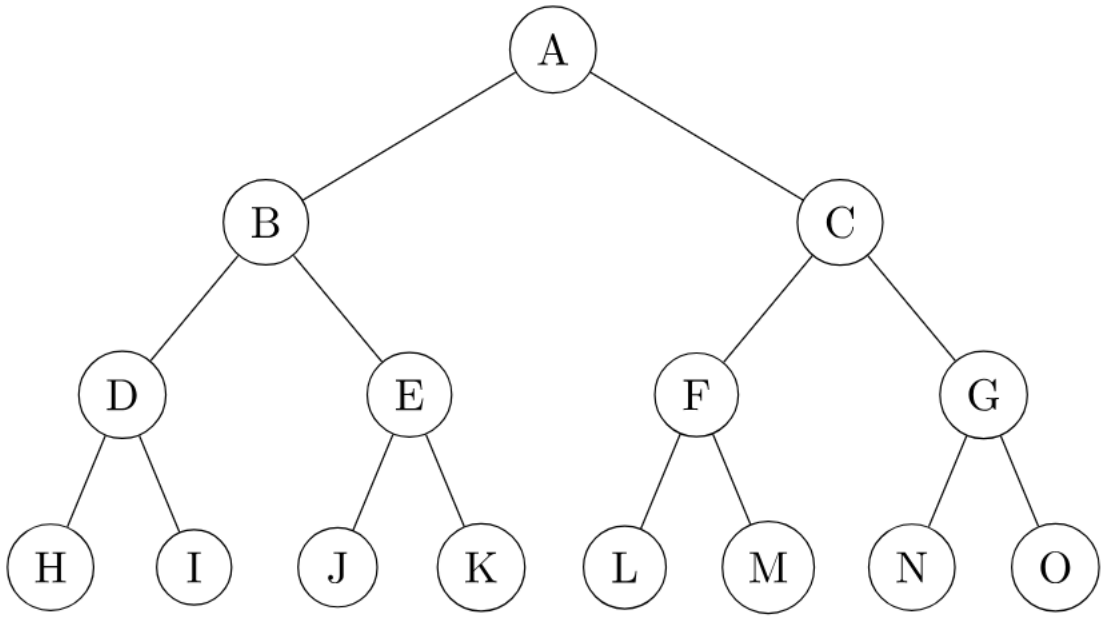
$$\log_B (X * Y) = \log_B X + \log_B Y$$

$$\log_B X^Y = Y * \log_B X$$

$$\log_A X = \frac{(\log_B X)}{(\log_B A)}$$

Mathematical background

- Binary tree (cây nhị phân)
  - Mỗi node có tối đa 2 node con
  - Mỗi node con chỉ có duy nhất một node cha (trừ node gốc)
- Cây nhị phân đầy đủ/hoàn chỉnh gồm 15 nodes bên dưới có bao nhiêu cấp (level)
  - $\lfloor \log_2 15 \rfloor + 1 = \lfloor 3.9 \rfloor + 1 = 4$



Mathematical background

- Probabilities (xác suất)
  - Có giá trị trong đoạn [0, 1]
  - 0: không bao giờ xảy ra (never)
  - 1: luôn luôn xảy ra (always)
- Nếu có N tình huống/sự kiện (possible situation) có thể xảy ra như nhau

$$p = 1/N$$

Mathematical background

- Summations (các phép tính tổng)

$$\sum_{i=1}^{N} C * i = C * \sum_{i=1}^{N} i$$

$$\sum_{i=1}^{N} (A + B) = \sum_{i=1}^{N} A + \sum_{i=1}^{N} B$$

$$\sum_{i=1}^{N} 1 = N$$

$$\sum_{i=C}^{N} i = \sum_{i=0}^{N-C} (i + C)$$

$$\sum_{i=0}^{N} (N - i) = \sum_{i=0}^{N} i$$

$$\sum_{i=1}^{N} C = C * N$$

$$\sum_{i=C}^{N} i = \sum_{i=0}^{N} i - \sum_{i=0}^{C-1} i$$

$$\sum_{i=1}^{N} i = \frac{N(N+1)}{2}$$

Mathematical background

$$\frac{N}{2}(N+1) = \frac{N(N+1)}{2}$$

$$\sum_{i=1}^{N} i^2 = \frac{N(N+1)(2N+1)}{6} = \frac{2N^3+3N^2+N}{6}$$

$$\sum_{i=0}^{N} 2^i = 2^{N+1} - 1$$

Mathematical background

$$\sum_{i=1}^{N} A^i = \frac{A^{N+1} - 1}{A - 1}, \quad \text{for some number } A$$

$$\sum_{i=1}^{N} i2^i = (N-1)2^{N+1} + 2$$

$$\sum_{i=1}^{N} \frac{1}{i} = \ln N$$

$$\sum_{i=1}^{N} \lg i \approx N \lg N - 1.5$$

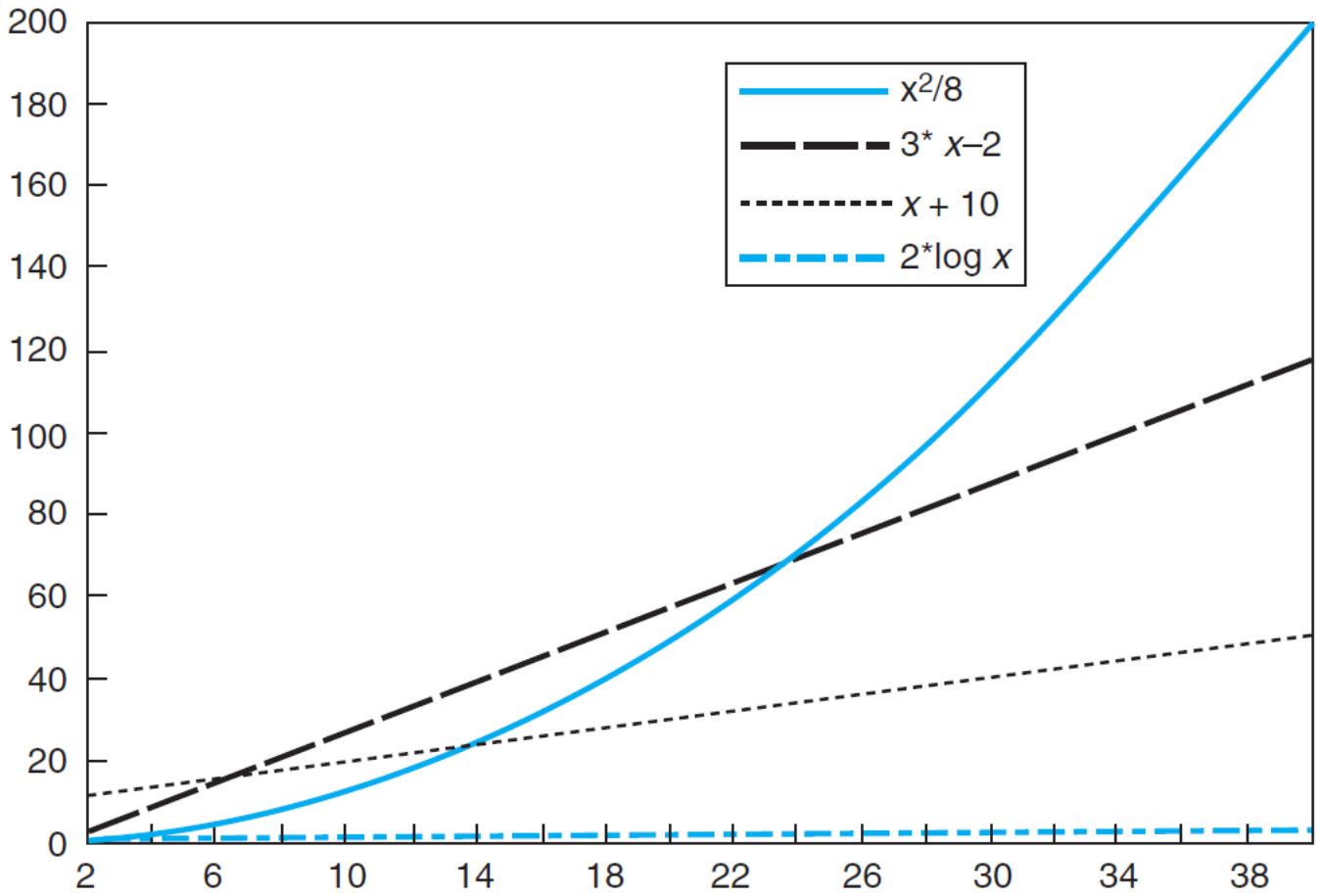
Rates of Growth

- Trong phân tích thuật toán, một trong các mối quan tâm chính là tốc độ gia tăng/phát triển (rate of increase) các thao tác tính toán (operations) của một thuật toán khi quy mô/kích thước của bài toán (input size) tăng lên.
  - Ví dụ để sắp xếp một mảng a gồm 1000 phần tử cần thời gian t_1
  - Nếu kích thước mảng a tăng lên 100,000 phần tử (gấp 100 lần) thì thời gian xử lý t_2 tăng gấp bao nhiêu lần so với t_1 ???

Rates of Growth

- Ý nghĩa
  - Gọi x là kích thước đầu vào của bài toán (input size)
  - f(x) là hàm số tính tổng số các thao tác tính toán khi thuật toán thực hiện.
  - Ví dụ nếu $f(x) = x^2$
- Nhận xét (theo đồ thị)
  - Khi x nhỏ thì f(x) tăng chậm so với các hàm khác.
  - Tuy nhiên f(x) sẽ tăng nhanh khi x tăng nhanh → điều này quan trọng. Tại sao?

- f(x) được xem như hàm tăng trưởng
- Chỉ quan tâm đến số hạng có bậc cao nhất (order)



Rates of Growth

	$\lg n$	n	$n \lg n$	n^2	n^3	2^n
1	0.0	1.0	0.0	1.0	1.0	2.0
2	1.0	2.0	2.0	4.0	8.0	4.0
5	2.3	5.0	11.6	25.0	125.0	32.0
10	3.3	10.0	33.2	100.0	1000.0	1024.0
15	3.9	15.0	58.6	225.0	3375.0	32768.0
20	4.3	20.0	86.4	400.0	8000.0	1048576.0
30	4.9	30.0	147.2	900.0	27000.0	1073741824.0
40	5.3	40.0	212.9	1600.0	64000.0	1099511627776.0
50	5.6	50.0	282.2	2500.0	125000.0	1125899906842620.0
60	5.9	60.0	354.4	3600.0	216000.0	1152921504606850000.0
70	6.1	70.0	429.0	4900.0	343000.0	1180591620717410000000.0
80	6.3	80.0	505.8	6400.0	512000.0	1208925819614630000000000.0
90	6.5	90.0	584.3	8100.0	729000.0	1237940039285380000000000000.0
100	6.6	100.0	664.4	10000.0	1000000.0	1267650600228230000000000000000.0

Classification of growth

- Big Omega
  - Mô tả cận dưới (lower bound) của độ phức tạp về thời gian/không gian của thuật toán → thời gian tối thiểu mà thuật toán thực hiện trong trường hợp tốt nhất.

$$T(n) = \Omega\big(f(n)\big) \text{ khi } T(n) \geq c \times f(n) \text{ với } c > 0$$

- Big Oh
  - Mô tả cận trên (upper bound) của độ phức tạp về thời gian/không gian của thuật toán → thời gian tối đa mà thuật toán thực hiện khi kích thước đầu vào tăng.

$$T(n) = \mathrm{O}\big(f(n)\big) \text{ khi } T(n) \leq c \times f(n) \text{ với } c > 0$$

  - Thường được sử dụng
- Big Theta
  - Mô tả cận trên và cận dưới của độ phức tạp về thời gian/không gian của thuật toán.

$$T(n) = \Theta\big(f(n)\big) \text{ khi } c_1 \times f(n) \leq T(n) \leq c_2 \times f(n) \text{ với } c_1, c_2 > 0$$

Classification of growth

- Công thức tính Big-Oh

$$g \in O(f) \text{ nếu } \lim_{n \to \infty} \frac{g(n)}{f(n)} = c \text{ với } c \in R^*$$

- [Tham khảo] toán học định nghĩa Big-Oh như sau:
  - Hàm $f(n)$ được gọi là $O(g(n))$ nếu tồn tại các hằng số c_0 và n_0 sao cho:

$$f(n) < c_0 * g(n) \text{ với mọi } n > n_0$$

  - Nói cách khác giá trị của hàm f(n) luôn nhỏ hơn giá trị của hàm g(n) nhân với một hằng số dương nào đó.

Classification of growth

- Ví dụ 1

$$f(n) = \frac{n(n+1)}{2}$$

  - $g(n) = n^2$
  - $c_0 = 1$
  - $n_0 = 1$

- Ta có, khi n > n_0

$$f(n) = \frac{n(n+1)}{2} = \left(\frac{n^2}{2} + \frac{n}{2}\right) < \left(\frac{n^2}{2} + \frac{n^2}{2}\right) = n^2 = 1 * g(n)$$

  - Để ý khi n > 1 → n^2 tăng nhanh hơn n

  → $f(n) < c_0 * g(n)$ khi n lớn hơn một giá trị n_0 nào đó.

- Khi này $f(n) = O\big(g(n)\big) = \mathrm{O}(n^2)$
  - Hay $\mathrm{O}(n^2)$ là cận trên của $f(n)$

Classification of growth

- Ví dụ 1

$$f(n) = \frac{n(n-1)}{2}$$

  - $g(n) = n^2$
  - $c_0 = 1/2$
  - $n_0 = 0$

- Ta có, khi n > n_0

$$f(n) = \frac{n(n-1)}{2} = \left(\frac{n^2}{2} - \frac{n}{2}\right) < \frac{n^2}{2} = \frac{1}{2}n^2 = \frac{1}{2} * g(n)$$

  → $f(n) < c_0 * g(n)$ khi n lớn hơn một giá trị n_0 nào đó.

  → Khi này $f(n) = O\big(g(n)\big) = O(n^2)$

- Các ví dụ trên là cơ sở để tính Big-Oh cho các thuật toán sau này.

Ví dụ về độ phức tạp và Big O

```c
// Ví dụ cơ bản về phân tích thuật toán & Big O Notation
#include <stdio.h>

// Đếm các cặp phần tử có tổng chia hết cho 5
// Bản chất như bài toán đếm các cặp đôi "hợp" tuổi :)
// Độ phức tạp: O(n^2)
int count_pairs(int a[], int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if ((a[i] + a[j]) % 5 == 0) {
                count++;
            }
        }
    }
    return count;
}
```

Ví dụ về độ phức tạp và Big O

- Đếm số lần thực hiện các block lệnh

```
// Đếm các cặp phần tử có tổng chia hết cho 5
// Bản chất như bài toán đếm các cặp đôi "hợp" tuổi :)
// Độ phức tạp: O(n^2)
int count_pairs(int a[], int n) {            // block A: 1 lần
    int count = 0;
    for (int i = 0; i < n; i++) {            // block B: N
        for (int j = i + 1; j < n; j++) {    // block C: N * (N - 1) / 2 ~ N^2
            if ((a[i] + a[j]) % 5 == 0) {
                count++;                     // block D: x lần (tùy thuộc input)
            }
        }
    }
    return count;
}
```

Ví dụ về độ phức tạp và Big O

Block	Time (sec)	Frequency	Total time
D	t_0	x (tùy thuộc input)	t_0x
C	t_1	$(N^2 - N) / 2$	$t_1(N^2 - N) / 2$
B	t_2	N	t_2N
A	t_3	1	t_3
Grand total		$(t_1/2)N^2 + (t_2 - t_1/2)N + t_3 + t_0x$	
Approximation		$\sim(t_1/2)N^2$ (giả sử x nhỏ)	
Order of growth		N^2	

Ví dụ về độ phức tạp và Big O

- $O(n^2)$ có ý nghĩa gì?
  - Giả sử mảng a gồm 100 phần tử, tổng thời gian chương trình xử lý là 0.1 sec
  - Nếu tăng kích thước mảng a lên 1000 phần tử (quy mô tăng 10 lần) → chương trình xử lý trong khoảng thời gian xấp xỉ là 0.1 sec x 10^2 = 10 sec
- Nếu bạn tìm được thuật toán B khác để giải quyết bài toán trên với O(n logn)

  →thuật toán B hiểu quả hơn vì O(n logn) < $O(n^2)$
  - Dù bạn có thực hiện thuật toán trên các máy tính khác nhau thì thời gian tăng xấp xỉ khi dữ liệu tăng theo cấp bình phương.

Thực hành

```c
// Một số ví dụ về Big O
#include <stdio.h>

// Tìm phần tử giữa mảng
// Độ phức tạp: T(n) = O(1)
int find_middle(int a[], int n) {
    int middle = n / 2;
    return a[middle];
}
```

```c
// Chia đôi một mảng
// Độ phức tạp: T(n) = O(log n)
void algorithm_1(int a[], int n) {
    int m = n / 2;
    while (m > 0) {
        printf("%d\n", a[m]);
        m = m / 2;
    }
}
```

- T(n) = O(1)
  - Giải thích?
  - Ý nghĩa?

- T(n) = O(log n)
  - Giải thích?
  - Ý nghĩa?

Thực hành

```
// Tìm phần tử lớn nhất trong mảng
// Độ phức tạp: T(n) = O(n)
int find_max(int a[], int n) {
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (a[i] > a[pos]) {
            pos = i;
        }
    }
    return a[pos];
}
```

Thực hành

```c
// In các phần tử trong ma trận 2 chiều
// Độ phức tạp: T(n) = O(n^2) hay O(m x n)
void print_matrix(int rows, int cols, int a[][cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }
}
```

- Quy tắc nhân O(m x n)

Thực hành

```c
// Độ phức tạp: T(n) = max(O(n), O(n^2)) = O(n^2)
// Quy tắc max
void algorithm_2(int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    printf("\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n - i; j++) {
            printf("%d ", a[j]);
        }
        printf("\n");
    }
}
```

- Quy tắc cộng
  - $O(n) + O(n^2)$
- Quy tắc max
  - $\max(O(n) + O(n^2)) = O(n^2)$
- Vậy kết hợp (combined complexity)
  - $O(n) + O(n^2) = O(n^2)$

Thực hành

- Độ phức tạp theo thời gian T(n) của thuật toán đệ quy tính giai thừa n!
  - T(n) = O(???)
- Tính
  - O(1):
  - T(n) = T(n-1) + O(1) // gọi đệ quy lần 1
  - T(n-1) = T(n-2) + O(1)
  - ...
  - T(2) = T(1) + O(1)
  - T(1) = O(1)
  - $\sum(\text{vế trái}) = \sum(\text{vế phải})$
  - T(n) = n * O(1) = O(n)

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```